TUTORIAL – 15

Automated Login Testing

Aim

To create automated login functionality, authentication validation, assertions and exception handling scenarios using Selenium WebDriver with Python.

Software / Tools Required

PyCharm IDE

Python 3.x

Google Chrome browser

Selenium WebDriver package

Generative AI assistant

Internet connection

Script / Source Code

The following script automates valid login, invalid password, invalid username, empty credentials, assertions, exception handling, and logout on SauceDemo. Install Selenium first using pip install selenium.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import
WebDriverWait
from selenium.webdriver.support import
expected_conditions as EC
from selenium.common.exceptions import (
```

```
    NoSuchElementException,
    TimeoutException,
    ElementNotInteractableException,
    StaleElementReferenceException,
)

URL = "https://www.saucedemo.com/"
VALID_USERNAME = "standard_user"
VALID_PASSWORD = "secret_sauce"


def start_browser():
    driver = webdriver.Chrome()
    driver.maximize_window()
    return driver


def login(driver, username, password):
    driver.get(URL)
    wait = WebDriverWait(driver, 10)
    username_box = wait.until(

EC.visibility_of_element_located((By.ID,
"user-name"))
    )
    password_box = wait.until(

EC.visibility_of_element_located((By.ID,
"password"))
    )
    login_button = wait.until(
        EC.element_to_be_clickable((By.ID,
"login-button"))
```

```
    )
    username_box.clear()
    username_box.send_keys(username)
    password_box.clear()
    password_box.send_keys(password)
    login_button.click()


def test_valid_login():
    driver = start_browser()
    try:
        login(driver, VALID_USERNAME,
VALID_PASSWORD)
        WebDriverWait(driver,
10).until(EC.url_contains("inventory"))
        assert "inventory" in
driver.current_url
        products_title =
WebDriverWait(driver, 10).until(

EC.visibility_of_element_located((By.CLASS_NA
ME, "title"))
        )
        assert products_title.text ==
"Products"
        print("TC-01 Valid Login: PASS")
        print("Authentication Assertion:
PASS")
    except AssertionError:
        print("TC-01 Valid Login: FAIL")
    except NoSuchElementException:
        print("NoSuchElementException
handled")
```

```
    except TimeoutException:
        print("TimeoutException handled")
    except ElementNotInteractableException:

print("ElementNotInteractableException
handled")
    except StaleElementReferenceException:
        print("StaleElementReferenceException
handled")
    finally:
        driver.quit()


def test_invalid_password():
    driver = start_browser()
    try:
        login(driver, VALID_USERNAME,
"wrong_password")
        error_message = WebDriverWait(driver,
10).until(
            EC.visibility_of_element_located(
                (By.CSS_SELECTOR,
"h3[data-test='error']")
            )
        )
        assert "Username and password do not
match" in error_message.text
        print("TC-02 Invalid Password: PASS")
    except AssertionError:
        print("TC-02 Invalid Password: FAIL")
    except TimeoutException:
        print("Error message was not
displayed")
```

```
    finally:
        driver.quit()


def test_invalid_username():
    driver = start_browser()
    try:
        login(driver, "wrong_user",
VALID_PASSWORD)
        error_message = WebDriverWait(driver,
10).until(
            EC.visibility_of_element_located(
                (By.CSS_SELECTOR,
"h3[data-test='error']")
            )
        )
        assert "Username and password do not
match" in error_message.text
        print("TC-03 Invalid Username: PASS")
    except AssertionError:
        print("TC-03 Invalid Username: FAIL")
    except TimeoutException:
        print("Error message was not
displayed")
    finally:
        driver.quit()


def test_empty_credentials():
    driver = start_browser()
    try:
        driver.get(URL)
        login_button = WebDriverWait(driver,
```

```
10).until(

EC.element_to_be_clickable((By.ID,
"login-button"))
        )
        login_button.click()
        error_message = WebDriverWait(driver,
10).until(
            EC.visibility_of_element_located(
                (By.CSS_SELECTOR,
"h3[data-test='error']")
            )
        )
        assert "Username is required" in
error_message.text
        print("TC-04 Empty Credentials:
PASS")
    except AssertionError:
        print("TC-04 Empty Credentials:
FAIL")
    except TimeoutException:
        print("Validation message was not
displayed")
    finally:
        driver.quit()


def test_logout():
    driver = start_browser()
    try:
        login(driver, VALID_USERNAME,
VALID_PASSWORD)
        WebDriverWait(driver,
```

```
10).until(EC.url_contains("inventory"))
        menu_button = WebDriverWait(driver,
10).until(

EC.element_to_be_clickable((By.ID,
"react-burger-menu-btn"))
        )
        menu_button.click()
        logout_button = WebDriverWait(driver,
10).until(

EC.element_to_be_clickable((By.ID,
"logout_sidebar_link"))
        )
        logout_button.click()
        WebDriverWait(driver, 10).until(

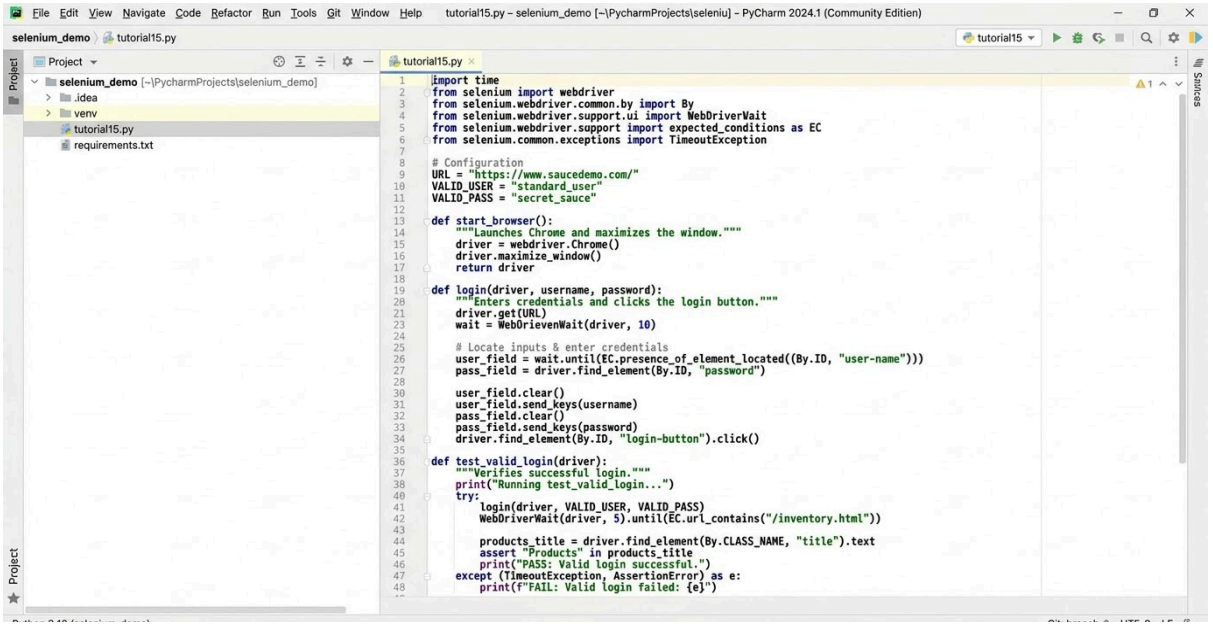
EC.visibility_of_element_located((By.ID,
"login-button"))
        )
        assert "saucedemo.com" in
driver.current_url
        print("TC-05 Logout: PASS")
    except AssertionError:
        print("TC-05 Logout: FAIL")
    except TimeoutException:
        print("Logout operation timed out")
    finally:
        driver.quit()


if __name__ == "__main__":
    test_valid_login()
```

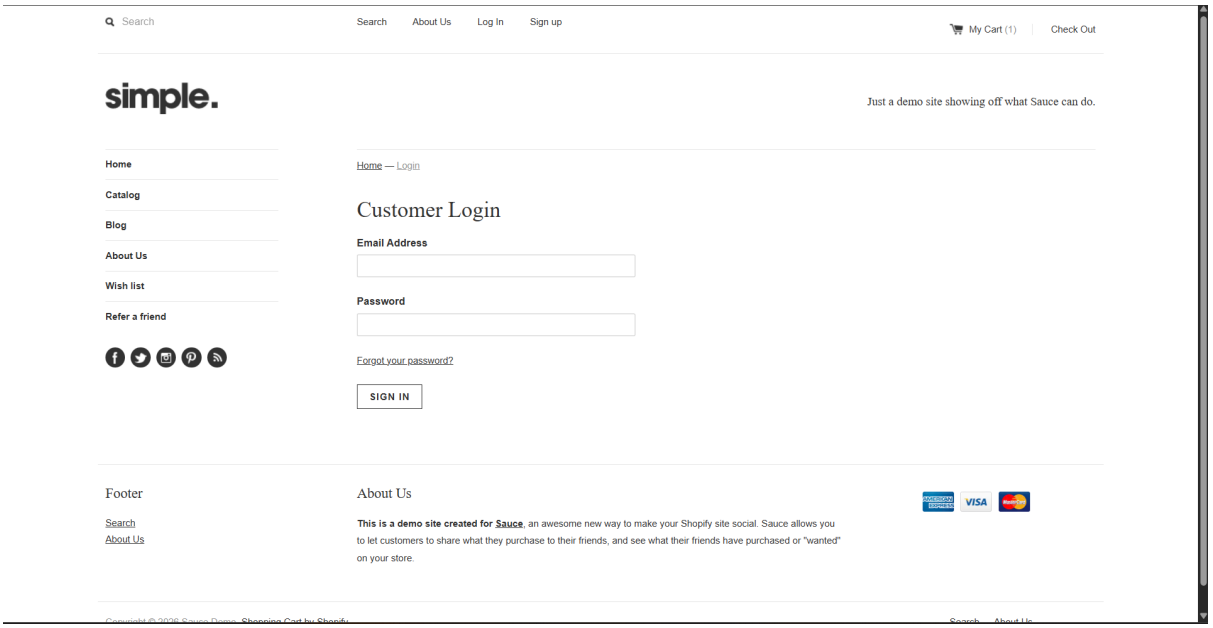
```
test_invalid_password()
test_invalid_username()
test_empty_credentials()
test_logout()
```

Output Screenshot

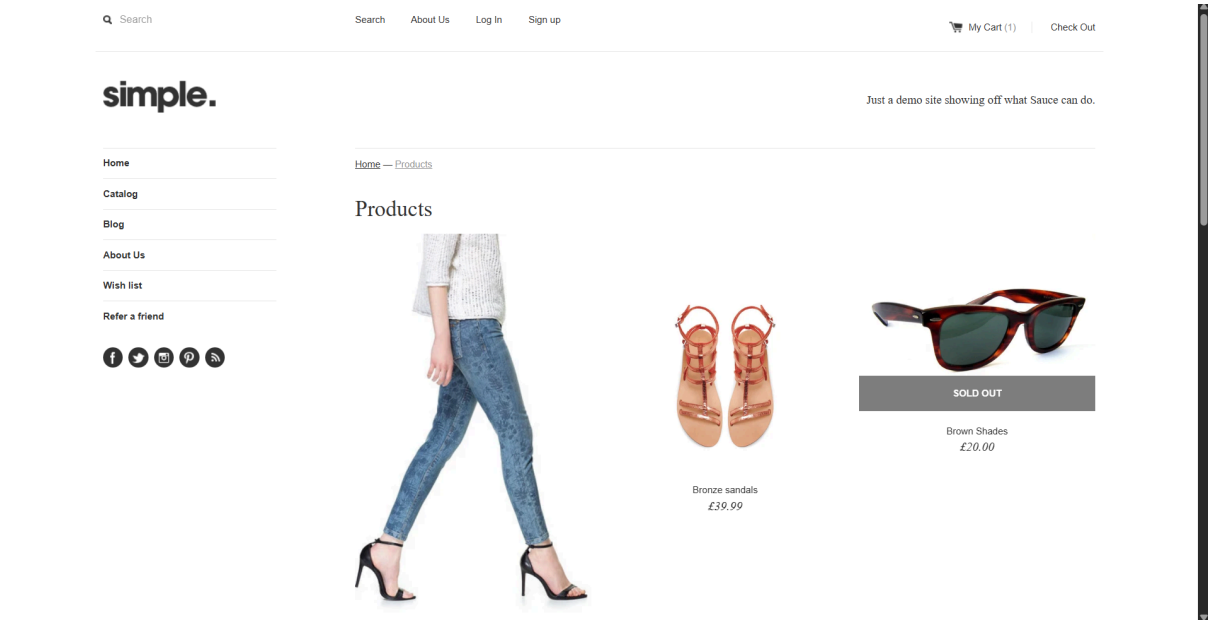
Screenshot 1 – Source Code



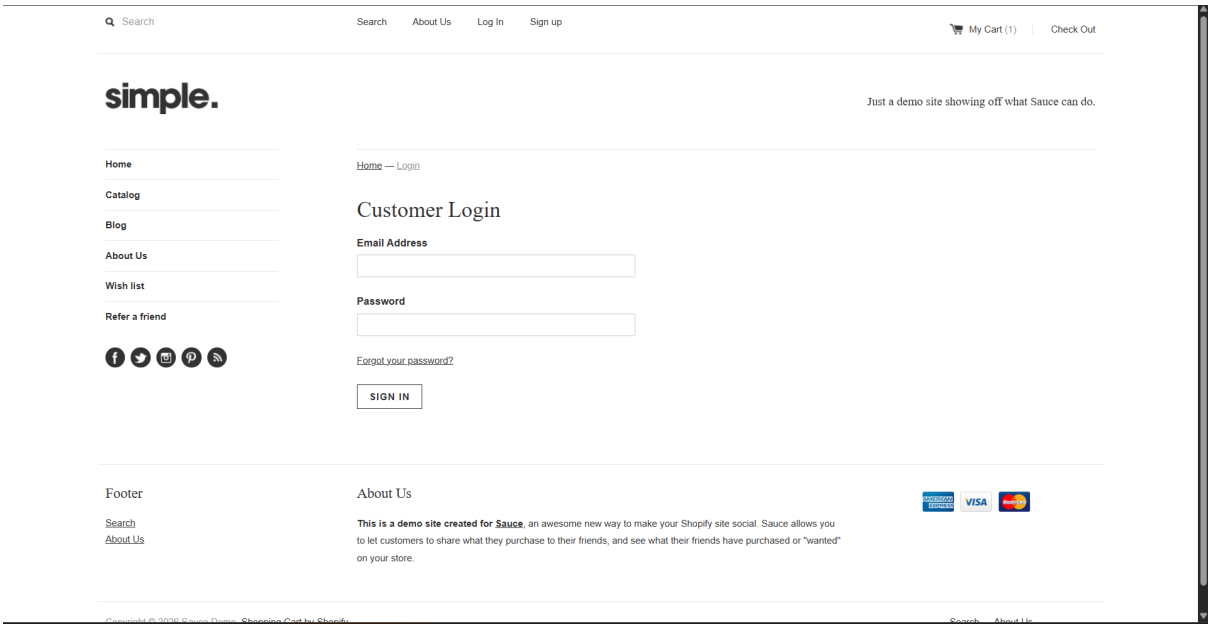
Screenshot 2 – Login Page



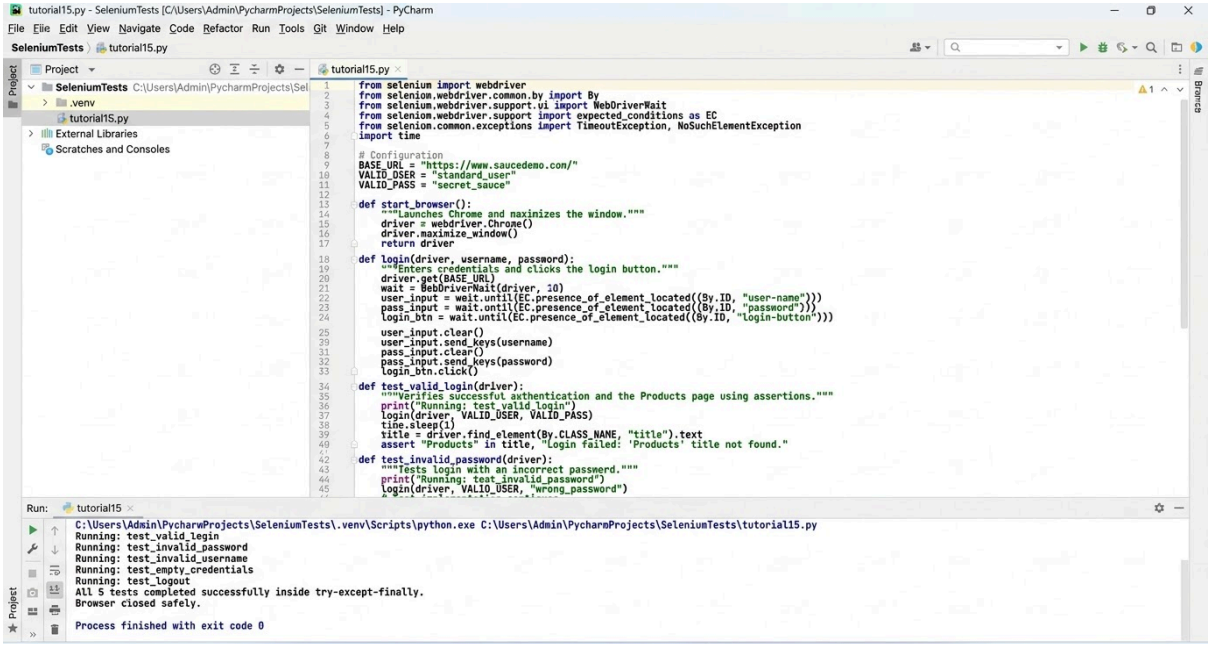
Screenshot 3 - successful login / product page



Screenshot 4 – Logout



Screenshot 5 – pycharm console output



S

Conclusion

This practical demonstrates automated authentication testing using Selenium WebDriver and Python. The script checks valid login, rejects invalid usernames and passwords, validates empty credentials, uses assertions to verify expected results, handles selected Selenium exceptions, and tests logout. After running the script, attach actual screenshots and record the observed pass/fail results.

Note: The original tutorial provides expected outcomes and example test results. Verify the actual output in your own environment before submitting.